SayOps — Build Progress Report

Narrate once, automate forever. · Status as of Fri 23 Oct 2026 (Day 3 of 10) · Submission deadline Fri 30 Oct 2026, 11:59 pm IST

2 / 10	45 / 45	2 / 2	~5 s	7 days
plan days complete	automated tests passing	live Groq compiles passed	compile time per session	left until deadline

Overall build progress: about 25% (core pipeline done; UI, sandbox, CLI and demo still to build)

1. Summary

The hardest part of SayOps — turning a recorded terminal session plus spoken reasons into a safe, cited automation plan — is built and works against the real model (Groq `openai/gpt-oss-120b`). On the SSL-renewal demo session it reliably catches the backup you said you'd take but didn't, turns “only renew if under 30 days” into a skip check, makes `nginx -t` abort and roll back, adds a rollback and a health check, and replaces hosts, domains and secrets with parameters. Nothing a judge can click exists yet: the web app is a placeholder page, and the script renderer, sandbox proof run, CLI and judge mode are next.

2. Progress against the 10-day plan

Day	Planned work	Status	Notes
1 · Wed 21	Monorepo, Next.js app, DB schema, SSL demo session, align + redact + tests	**Partial**	All code done. Not yet deployed (needs Vercel + Neon accounts).
2 · Thu 22	AI compile, citations, danger radar, plan versions	**Partial**	Compile, validation, radar and Said-vs-Did done and verified live. Saving plan versions needs DATABASE_URL.
3 · Fri 23	Renderers (script, workflow, runbook) + session page, Said-vs-Did banner	**Not started**	Next up.
4 · Sat 24	Docker sandbox + GitHub Actions proof run, broken-config fixture	**Not started**	Will also fix the cp -a backup/restore issue (section 6).
5 · Sun 25	CLI: sayops start/stop, zsh hooks, why, upload	**Not started**	Week-1 check-in due before Mon 26 Oct.
6 · Mon 26	Talk-to-fix corrections + diff	**Not started**	Compiler already accepts corrections (X ids).
7 · Tue 27	Real chore recorded + toil meter + 2nd seed	**Not started**	Needs your real chore.
8 · Wed 28	Judge mode polish, rate limits, cached runs	**Not started**	
9 · Thu 29	Tests, README, demo video	**Not started**	
10 · Fri 30	Usage card, about text, submit	**Not started**	

3. What is built

Component	File	What it does	Status
Data formats	packages/core/src/schemas.ts	Session events (commands, narrations) and the plan format; strict JSON Schema for Groq	**Done**
Alignment	align.ts	Attaches each spoken reason to the command run before it; drops typo-then-retry commands	**Done**
Secret redaction	redact.ts	Masks tokens, keys, passwords, connection strings and high-entropy strings before upload	**Done**
AI compile	compile.ts, llm.ts	Prompt + Groq call; one retry with exact errors; rate-limit wait; truncation handling	**Done**
Plan checks	validate.ts	Every step cites a real source; no hard-coded hosts or secrets; spoken conditions must gate a step; no orphan restores	**Done**
Said-vs-Did	saidvsdid.ts	Deterministic backstop: “I always back up” with no backup command is always reported	**Done**
Danger radar	radar.ts	Fixed rules for rm -rf, DROP, kubectl delete, terraform destroy, systemctl stop…; only raises risk	**Done**
SSL demo + reference plan	seeds/ssl-renew*.ts	Judge-mode demo session and its cached plan	**Done**
Live compile script	scripts/compile-seed.ts	npm run compile:seed runs the real model and checks 8 demo-critical properties	**Done**
Database schema	web/db/schema.ts	Sessions, commands, narrations, plan versions, findings, artifacts, proof runs, corrections	**Partial**
Web app	web/app/page.tsx	Placeholder page rendering the aligned demo session	**Partial**
Renderers	—	Bash script, GitHub workflow, Markdown runbook	**Not started**
Proof runner	—	Docker sandbox in GitHub Actions with per-step results	**Not started**
CLI	—	sayops start / why / stop, zsh hooks, upload	**Not started**
Judge mode	—	Landing, session page, Said-vs-Did banner, try-it, reset	**Not started**

4. Live result on the SSL demo session

Two back-to-back live runs both passed every check on the first attempt (about 5 seconds and 4,200 tokens each, under the free tier's 8,000 tokens/minute). This is the plan the model produced:

Step	Command (run on $HOST)	Comes from	On failure	Risk
S1	`sudo certbot certificates -d $DOMAIN` *skip whole job if:* `openssl x509 -checkend 2592000 …`	C1, N1 + N2 “only renew if under 30 days”	abort	LOW
S2	`sudo systemctl stop nginx`	C3, N3 “certbot needs port 80”	abort	MEDIUM
S3	`sudo cp -a /etc/letsencrypt/live/$DOMAIN …bak.$SAYOPS_RUN_ID`	N4 only — ADDED by Said-vs-Did	abort	LOW
S4	`sudo certbot renew --cert-name $DOMAIN`	C4, N4	abort	MEDIUM
S5	`sudo nginx -t`	C5, N5 “always test the config”	abort + rollback	LOW
S6	`sudo systemctl start nginx`	C6, N6	abort	MEDIUM

Rollback: restore the backup, start nginx. **Verify:** the site returns HTTP 200. **Parameters:** HOST (web-1), DOMAIN (shop.example.com), AUTH_TOKEN (was redacted). **Finding:** said_not_done — “Before this I always back up the old cert folder” but no backup ran.

Demo-critical checks (both live runs)

Check	Result
Said-vs-Did catches the missing backup	Pass
Backup step added automatically	Pass
nginx -t guard stops the run	Pass
30-day skip condition	Pass
Rollback present	Pass
Health check present	Pass
HOST / DOMAIN parameters	Pass
Same step count as reference plan	Pass

5. Problems found and fixed during live testing

Problem with the model's output	Fix (deterministic, not left to the AI)
Reply cut off before the plan was complete	Raised the reply limit; a cut-off reply now uses the retry instead of crashing
Hit the free tier's per-minute token limit	Automatic wait using Groq's retry-after; lower reasoning effort
Backup habit read as a plain reason, so no Said-vs-Did finding	saidvsdid.ts detects the claim and feeds it back; reports it itself if missed twice
Redacted token left as <REDACTED:bearer> in the script	Validation rejects it; must become a parameter
30-day rule turned into a normal step	A spoken condition must gate a step, or the plan is sent back
nginx -t guard did not trigger rollback	Guarded steps always abort and roll back
IDs drifted (step1, commands listed as narrations)	ID formats described in the schema; near-miss IDs normalised

6. Known issues and decisions

- **Backup/restore commands:** the model uses `cp -a dir dir.bak`; restoring onto an existing folder copies *into* it. To fix on Day 4; the sandbox proof run is designed to catch exactly this.
- **Free-tier limit:** 8,000 tokens/minute fits one compile per minute. Switch Groq to pay-as-you-go before judging; demo plans will be cached.
- **Decisions:** LLM = Groq; hosting = Vercel + Neon + GitHub Actions ($0); narration command is `why` (macOS already has `say`); a narration explains the command run before it.
- **Not committed yet:** the repo has no commits yet; make the first commit with a message dictated through Wispr Flow.

7. Waiting on you

Item	Why it matters	Status
Wispr Flow Pro (WEMAKEDEVS)	Free month; renews Nov 21 at Rs 400/mo unless cancelled	**Done**
Register on wemakedevs.org/wispr (Builders) + first Flow	Required to enter	**Not started**
Dictate all messages, commits, docs with fn	Judged on usage card; most-words prize	**Not started**
Week-1 check-in + #NoHandsChallenge post	Due before Mon 26 Oct	**Not started**
Groq API key in sayops/.env	Live compile	**Done**
Neon DATABASE_URL	Saving sessions and plan versions	**Not started**
GitHub repo + Vercel project	Deploy, proof runs, judges' link	**Not started**
One real chore you do by hand	Required “real result” (Day 7)	**Not started**

8. Next steps

- **Day 3:** render the plan into a lint-clean bash script (with set -euo pipefail, rollback trap, skip checks, quoted reasons as comments), a GitHub Actions workflow and a Markdown runbook; build the session page with replay timeline and Said-vs-Did banner.
- **Day 4:** Docker sandbox (nginx + fake certbot) and a GitHub Actions proof run showing green steps, plus a broken-config run that proves rollback.
- **Day 5:** the CLI so a real recording appears in the web app.